

Node.js na Prática: do código ao deploy na nuvem

Construindo uma aplicação CRUD completa com JavaScript no back-end — ferramentas, versionamento com Git e GitHub, deploy na AWS e boas práticas de qualidade.



AUTORES

Matheus Albuquerque Bezerra Porto

João Gabriel Sousa e Silva

Felipe Diniz Mariano

Thiago Yan De Paula Lima

Rafaele Gomes

Paulo Haskley Queiroz de Lima

Sumário

01 Introdução ao Node.js

O que é, como funciona, história e onde é usado

02 Ferramentas de Desenvolvimento

VS Code, npm, Express, Supabase e configuração do ambiente

03 Passo a Passo: Construindo a Aplicação

Uma API REST de produtos com front-end web

04 Versionamento com Git e GitHub

Commits, branches, pull requests e boas práticas

05 Deploy na Nuvem (AWS EC2)

Publicando o front-end e o back-end em máquinas virtuais

06 Testes e Qualidade de Software

Jest, Supertest, ESLint e integração contínua

07 Conclusão e Reflexão

Desafios enfrentados e aprendizados da equipe

Introdução ao Node.js

E se a mesma linguagem que dá vida às páginas web pudesse também rodar no servidor? Essa é a ideia por trás do Node.js.

O que é o Node.js?

O **Node.js** é um ambiente de execução (*runtime*) que permite rodar JavaScript fora do navegador. Antes dele, o JavaScript era uma linguagem exclusiva do front-end: servia para criar interatividade nas páginas, mas não para construir servidores, acessar bancos de dados ou manipular arquivos. O Node.js mudou isso ao levar o motor **V8** — o mesmo que executa JavaScript dentro do Google Chrome — para qualquer computador ou servidor.

Na prática, isso significa que um desenvolvedor pode usar **uma única linguagem para todo o sistema**: o JavaScript escreve tanto a interface que o usuário vê quanto a lógica que roda no servidor. Foi exatamente essa a abordagem usada no projeto deste ebook.

Um pouco de história

O Node.js foi criado em **2009 por Ryan Dahl**, que estava insatisfeito com a forma como os servidores web da época lidavam com muitas conexões simultâneas. A solução dele foi um modelo baseado em **eventos e operações não bloqueantes**: em vez de criar um processo pesado para cada usuário conectado, o Node.js usa um laço de eventos (*event loop*) que atende milhares de requisições de forma assíncrona, com baixo consumo de memória.

Em 2010 surgiu o **npm** (Node Package Manager), o gerenciador de pacotes que hoje é o maior ecossistema de bibliotecas de software do mundo, com milhões de pacotes prontos para uso. Em 2015, a comunidade se reorganizou sob a *Node.js Foundation* (hoje parte da OpenJS Foundation), garantindo a evolução aberta e contínua da plataforma.

Como ele funciona?

O coração do Node.js é o **event loop**. Imagine um restaurante com um único garçom extremamente eficiente: ele anota o pedido da mesa 1, e *enquanto a cozinha prepara o prato*, já vai atender as mesas 2 e 3 — em vez de ficar parado esperando. Quando a cozinha avisa que o prato está pronto, ele o entrega. O Node.js trabalha da mesma forma: enquanto espera uma resposta do banco de dados, ele já atende outras requisições.

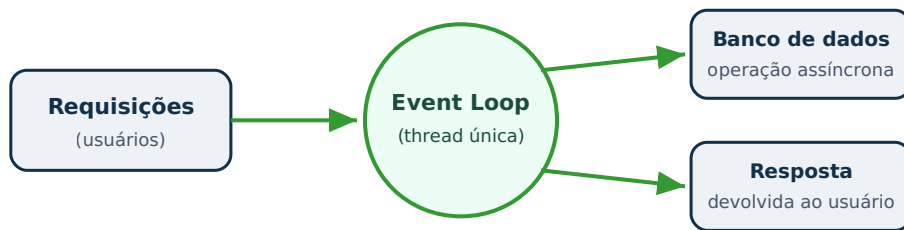


Figura 1.1 — O event loop do Node.js. *Descrição alternativa: diagrama com três blocos; requisições de usuários entram em um círculo central chamado event loop, que delega tarefas demoradas (como consultas ao banco) e devolve respostas sem bloquear o atendimento.*

Vantagens e onde é mais utilizado

- **Uma linguagem só:** JavaScript no front-end e no back-end reduz a curva de aprendizado da equipe.
- **Alta performance em I/O:** ideal para APIs, chats e aplicações em tempo real com muitas conexões.
- **Ecossistema gigante:** o npm oferece pacotes prontos para quase tudo (servidores, autenticação, banco de dados).
- **Comunidade ativa:** usado por Netflix, Uber, LinkedIn, PayPal e milhares de startups.

Os usos mais comuns incluem **APIs REST** (caso deste projeto), microsserviços, aplicações em tempo real (chats, jogos), ferramentas de linha de comando e automações.

Ferramentas de Desenvolvimento

Um bom ambiente de desenvolvimento é como uma oficina organizada: cada ferramenta tem seu papel e o trabalho flui melhor.

As ferramentas do projeto

Ferramenta	Categoria	Papel no projeto
VS Code	Editor / IDE	Escrita do código, com extensões para JavaScript e Git integrado
Node.js + npm	Runtime e gerenciador	Executa o servidor e instala as bibliotecas
Express	Framework web	Cria as rotas da API REST de forma simples
Supabase	Banco de dados	PostgreSQL gerenciado na nuvem, com painel visual
Git + GitHub	Versionamento	Histórico de alterações e trabalho em equipe
Postman	Teste de API	Testa as rotas (GET, POST, PUT, DELETE) antes do front-end existir
AWS EC2	Nuvem	Máquinas virtuais onde a aplicação foi publicada

Bibliotecas utilizadas

- `express` — framework minimalista que organiza as rotas HTTP da API;
- `@supabase/supabase-js` — cliente oficial para conversar com o banco PostgreSQL;
- `cors` — libera o acesso à API a partir de outros endereços (essencial porque front e back rodam em servidores diferentes);
- `dotenv` — carrega senhas e chaves de um arquivo `.env`, mantendo segredos fora do código.

Configurando o ambiente do zero

Para reproduzir o ambiente do projeto, basta seguir quatro passos:

Passo 1 — Instalar o Node.js. Baixe a versão LTS em `nodejs.org` e confirme a instalação no terminal:

```
# Verifica as versões instaladas
node -v # ex.: v22.14.0
npm -v # ex.: 10.9.2
```

Passo 2 — Criar o projeto. O comando `npm init` gera o `package.json`, a "certidão de nascimento" do projeto, que registra nome, scripts e dependências:

```
mkdir backend && cd backend
npm init -y
```

Passo 3 — Instalar as dependências.

```
npm install express cors dotenv @supabase/supabase-js
```

Passo 4 — Criar o arquivo de variáveis de ambiente.

```
.env
SUPABASE_URL=https://seuprojeto.supabase.co
SUPABASE_KEY=sua-chave-secreta
PORT=3000
```

⚠ **Atenção:** o arquivo `.env` guarda segredos e **nunca** deve ser enviado ao GitHub. Ele precisa estar listado no `.gitignore` (veremos isso no Capítulo 4).

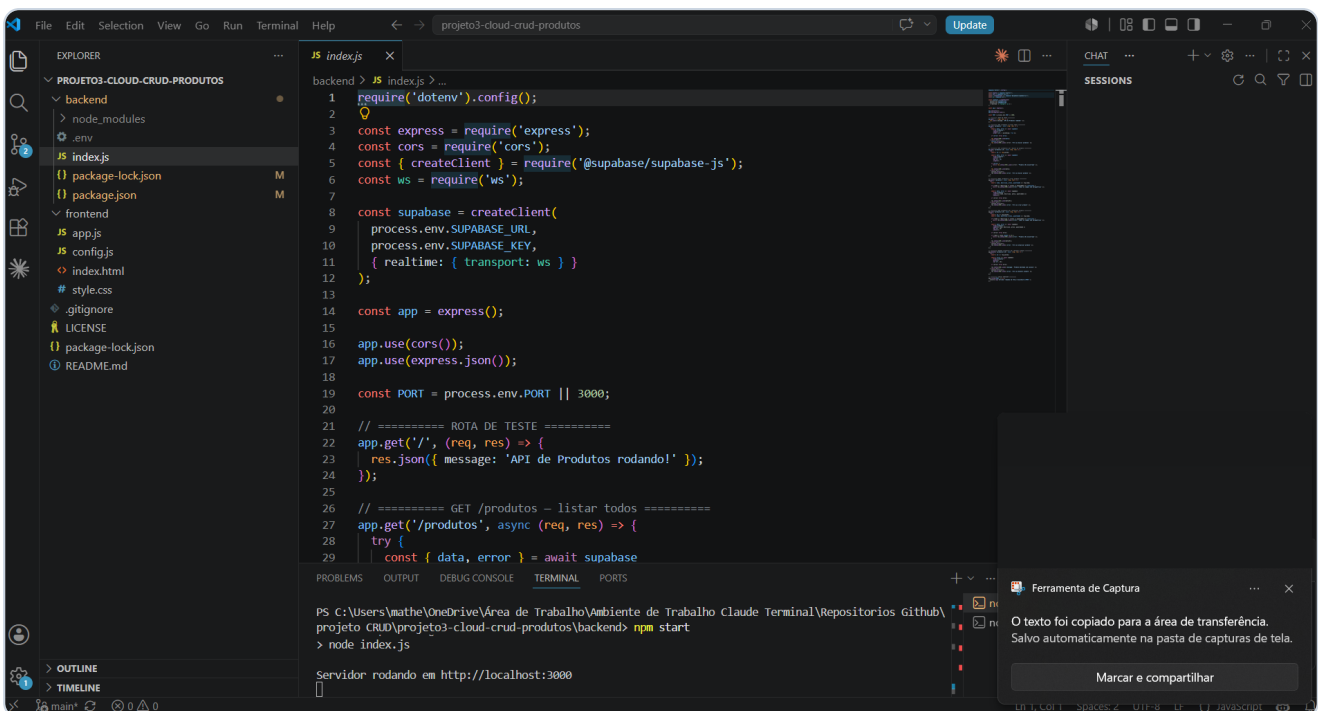


Figura 2.1 — Ambiente de desenvolvimento real do projeto. Descrição alternativa: editor VS Code exibindo a árvore de arquivos do projeto (pastas backend e frontend), o código do index.js aberto e o terminal integrado com a mensagem "Servidor rodando em http://localhost:3000".

Passo a Passo: Construindo a Aplicação

A aplicação desenvolvida é um **CRUD de produtos**: um sistema que permite Criar, Listar (Read), Atualizar (Update) e Deletar produtos de um estoque.

A arquitetura

O sistema foi dividido em três partes independentes, cada uma rodando em um lugar diferente — uma arquitetura comum em aplicações reais na nuvem:



Figura 3.1 — Arquitetura do projeto. Descrição alternativa: fluxo da esquerda para a direita — o navegador do usuário acessa o front-end hospedado em uma máquina virtual; o front-end consome a API REST em Node.js hospedada em outra máquina virtual; a API consulta o banco PostgreSQL gerenciado pelo Supabase.

O banco de dados

No Supabase foi criada a tabela **produtos**, com as colunas: **id** (chave primária com incremento automático), **nome**, **descricao**, **preco**, **quantidade** e **created_at** (data de criação preenchida automaticamente).

id	nome	descricao	preco	quantidade	created_at
1	Notebook Dell Inspiron 15	i7, 16GB RAM, SSD 512GB	4500.00	10	2026-06-10 14:25:25.364628+0
2	Mouse Logitech MX Master 3	Mouse sem fio ergonômico	549.90	25	2026-06-10 14:25:25.364628+0
3	Teclado Mecânico Keychron K2	Switch brown, bluetooth e cabo	499.00	15	2026-06-10 14:25:25.364628+0

Figura 3.2 — A tabela **produtos** no painel do Supabase. Descrição alternativa: editor de tabelas do Supabase mostrando as colunas **id**, **nome**, **descricao**, **preco**, **quantidade** e **created_at**, com três produtos cadastrados.

O servidor Express

O arquivo principal do back-end, `index.js`, começa carregando as configurações, conectando ao Supabase e ligando os middlewares:

```
backend/index.js
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const { createClient } = require('@supabase/supabase-js');

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_KEY
);

const app = express();
app.use(cors()); // permite acesso do front-end
app.use(express.json()); // entende requisições em JSON

const PORT = process.env.PORT || 3000;
```

As rotas da API

Cada operação do CRUD vira uma rota HTTP. A listagem de produtos, por exemplo, responde a `GET /produtos`:

```
backend/index.js - listar todos
app.get('/produtos', async (req, res) => {
  try {
    const { data, error } = await supabase
      .from('produtos')
      .select('*')
      .order('id', { ascending: true });

    if (error) throw error;
    res.status(200).json(data);
  } catch (err) {
    res.status(500).json({ error: 'Erro ao buscar produtos' });
  }
});
```

E a criação de um produto responde a `POST /produtos`, validando os campos antes de gravar:

```

backend/index.js – criar produto
app.post('/produtos', async (req, res) => {
  const { nome, descricao, preco, quantidade } = req.body;

  if (!nome || !descricao || !preco || quantidade === undefined) {
    return res.status(400).json({ error: 'Todos os campos são obrigatórios' });
  }

  const { data, error } = await supabase
    .from('produtos')
    .insert([ { nome, descricao, preco, quantidade } ])
    .select();

  res.status(201).json(data[0]);
});

```

O mesmo padrão se repete para `GET /produtos/:id` (buscar um), `PUT /produtos/:id` (atualizar) e `DELETE /produtos/:id` (excluir) — completando as cinco rotas do CRUD.

Método	Rota	Ação	Resposta de sucesso
GET	/produtos	Lista todos os produtos	200 + lista em JSON
GET	/produtos/:id	Busca um produto	200 + produto (404 se não existir)
POST	/produtos	Cria um produto	201 + produto criado
PUT	/produtos/:id	Atualiza um produto	200 + produto atualizado
DELETE	/produtos/:id	Exclui um produto	200 + mensagem de confirmação

O front-end

A interface foi construída com **HTML, CSS e JavaScript puros**, sem frameworks — uma escolha proposital para mostrar que o consumo de uma API depende apenas da função nativa `fetch`:

```

frontend/app.js – consumindo a API
async function listarProdutos() {
  const res = await fetch(`${API_URL}/produtos`);
  const produtos = await res.json();
  renderizarTabela(produtos);
}

```

A tela reúne um formulário de cadastro/edição, uma busca por ID e uma tabela com ações de editar e excluir em cada linha — cobrindo visualmente as quatro operações do CRUD.

❏ **O que é CORS?** Por segurança, o navegador bloqueia requisições de uma página para servidores em endereços diferentes do dela (a chamada *política de mesma origem*). Como nosso front-end e nossa API rodam em servidores separados, o back-end precisa declarar explicitamente "este acesso é permitido" — e é exatamente isso que o middleware `cors` faz, enviando cabeçalhos de autorização (*Cross-Origin Resource Sharing*) em cada resposta. Sem ele, o navegador recusaria os dados e a lista de produtos jamais carregaria.

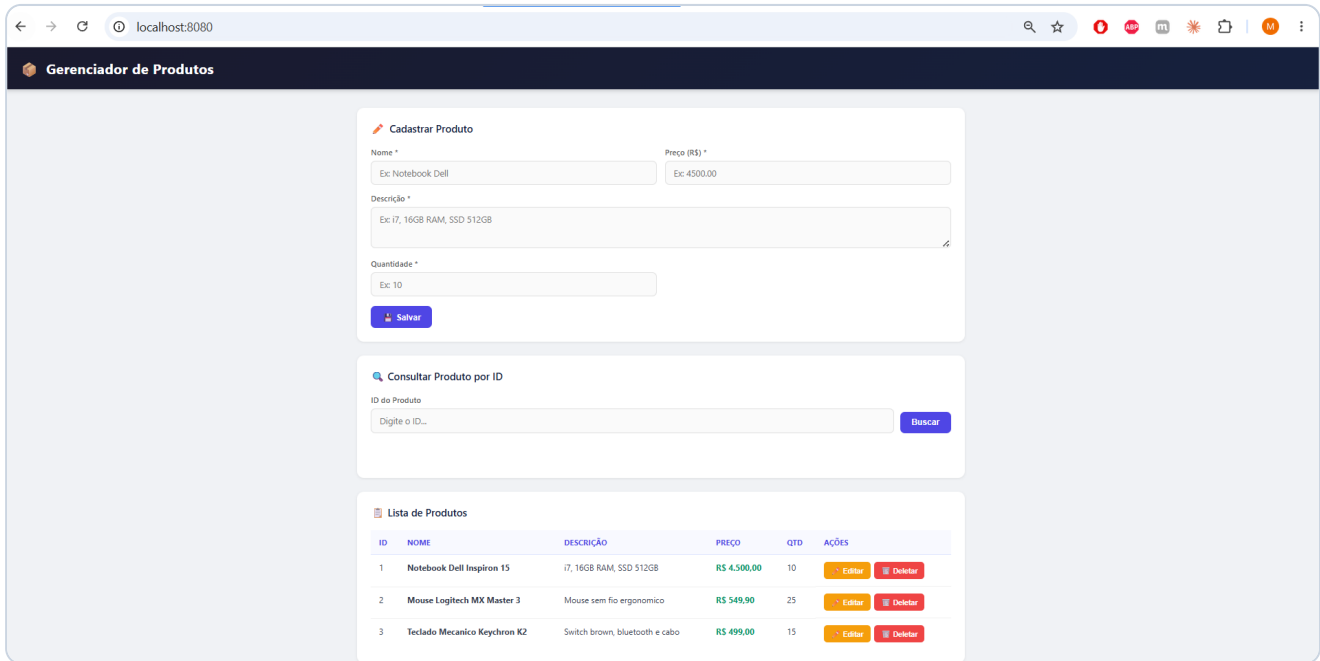


Figura 3.3 — A aplicação em execução no navegador. *Descrição alternativa: página web do Gerenciador de Produtos com formulário de cadastro no topo, busca por ID e tabela listando três produtos com botões de editar e deletar.*

Versionamento com Git e GitHub

O Git é a "máquina do tempo" do projeto: cada mudança fica registrada, pode ser revisada e, se necessário, desfeita. O GitHub é onde esse histórico vive na nuvem e onde a equipe colabora.

Configuração inicial

```
# Identifica o autor dos commits (uma vez só por máquina)
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"

# Inicia o repositório dentro da pasta do projeto
git init
```

O fluxo de trabalho do Git

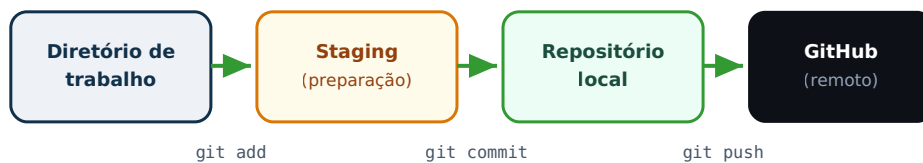


Figura 4.1 — Fluxo básico do Git. Descrição alternativa: quatro caixas em sequência — diretório de trabalho, área de staging, repositório local e GitHub — conectadas pelos comandos `git add`, `git commit` e `git push`.

```
# Fluxo usado a cada alteração no projeto
git add backend/index.js           # prepara o arquivo
git commit -m "feat: backend e frontend completos"
git push origin main               # envia para o GitHub
```

Commits semânticos (Conventional Commits)

A equipe adotou o padrão **Conventional Commits**, em que cada mensagem começa com um prefixo que indica o tipo da mudança. Isso torna o histórico legível como uma linha do tempo. Veja o histórico real do projeto:

```
git log --oneline (histórico real do projeto)
8140f98 fix(backend): adiciona ws para compatibilidade com Node < 22
c0467cf feat: backend e frontend completos
0971c16 Merge pull request #1 from matheusabporto/feat/setup-supabase
38d7572 docs(backend): documenta schema da tabela produtos
eed319d chore: estrutura inicial do projeto
ee5c7ab Initial commit
```

- **feat:** nova funcionalidade
- **fix:** correção de bug
- **docs:** documentação
- **chore:** tarefas de manutenção.

Branches e Pull Requests

Em vez de alterar o código principal diretamente, criamos uma **branch** (ramificação) para cada funcionalidade. No projeto, a configuração do banco foi feita na branch **feat/setup-supabase** e integrada à **main** por meio de um **Pull Request** — o pedido formal de revisão e fusão do código, visível no histórico acima (commit **0971c16**):

```
git checkout -b feat/setup-supabase # cria e muda para a branch
# ... trabalho e commits ...
git push origin feat/setup-supabase # publica a branch
# No GitHub: abrir Pull Request → revisar → Merge
```

Boas práticas adotadas

- **Commits pequenos e frequentes**, cada um com um propósito único;
- **Mensagens no padrão semântico**, descrevendo o "o quê" e o "porquê";
- **.gitignore bem configurado**: **node_modules/** (pesada e reinstalável) e **.env** (segredos) ficam fora do repositório;
- **Uma branch por funcionalidade**, integrada via Pull Request.

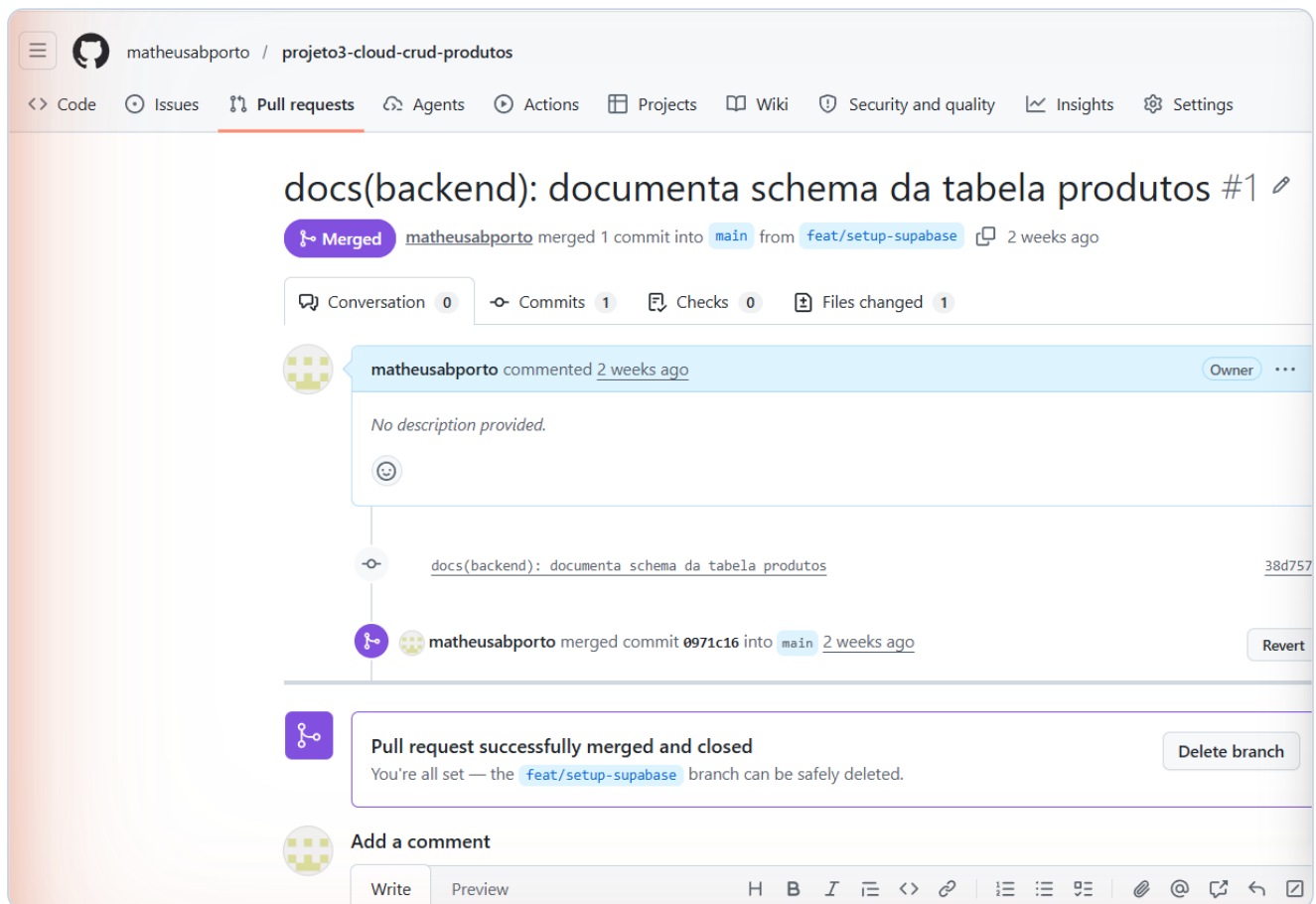


Figura 4.2 — O Pull Request #1 do projeto, mesclado na branch principal. *Descrição alternativa: página do GitHub exibindo o pull request da branch feat/setup-supabase com o selo roxo "Merged" e a mensagem "Pull request successfully merged and closed".*

Deploy na Nuvem (AWS EC2)

*Desenvolver é metade do caminho; a outra metade é fazer a aplicação existir na internet. O deploy foi feito na **AWS**, usando duas máquinas virtuais EC2 — uma para o front-end e outra para o back-end.*

Por que duas máquinas?

Separar front-end e back-end em servidores diferentes espelha a arquitetura de sistemas profissionais: cada parte pode ser atualizada, escalada ou reiniciada sem afetar a outra. O banco de dados, por sua vez, nem precisa de servidor próprio — o **Supabase** o oferece como serviço gerenciado (DBaaS), cuidando de backups e disponibilidade.

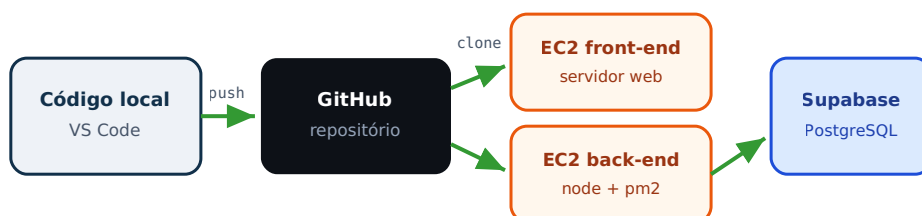


Figura 5.1 — Fluxo de deploy. Descrição alternativa: o código sai do computador local para o GitHub via `git push`; de lá é clonado nas duas máquinas EC2 (front-end e back-end); o back-end conecta-se ao banco PostgreSQL do Supabase.

Passo a passo do deploy do back-end

- 1. Criar a instância EC2** no console da AWS (Amazon Linux 2023, tipo t3.micro — elegível no nível gratuito) e configurar o **Security Group** liberando as portas `22` (SSH) e `3000` (API).
- 2. Conectar à VM** — via SSH com a chave `.pem` gerada na criação, ou pelo **EC2 Instance Connect**, o terminal no próprio navegador (botão "Conectar" no console):

```
ssh -i "chave-projeto.pem" ec2-user@IP_PUBLICO_DA_VM
```

- 3. Instalar o Node.js e clonar o projeto** dentro da VM:

```
# Instala o Node.js e o Git
sudo dnf install -y nodejs git

# Baixa o código e instala as dependências
git clone https://github.com/matheusabporto/projeto3-cloud-crud-produtos.git
cd projeto3-cloud-crud-produtos/backend
npm install

# Cria o .env com as credenciais do Supabase
nano .env
```

4. Manter o servidor no ar com PM2 — sem ele, a API morreria ao fechar o terminal:

```
sudo npm install -g pm2
pm2 start index.js --name api-produtos
pm2 startup && pm2 save # reinicia junto com a VM
```

Deploy do front-end

Na segunda VM, o processo é ainda mais simples: clonar o repositório e servir a pasta **frontend** com o **nginx**, um servidor web estático. Antes disso, um detalhe crucial — apontar o front-end para o IP público do back-end:

```
frontend/config.js
// Troque para o IP público da EC2 do backend antes do deploy
const API_URL = 'http://IP_PUBLICO_DO_BACKEND:3000';
```

```
# Na VM do front-end (porta 80 liberada no Security Group)
sudo dnf install -y nginx git
git clone https://github.com/matheusabporto/projeto3-cloud-crud-produtos.git
sudo cp -r projeto3-cloud-crud-produtos/frontend/* /usr/share/nginx/html/
sudo systemctl enable --now nginx
```

□ **Alternativas de deploy:** para projetos menores, plataformas como **Vercel** e **Render** automatizam tudo isso a partir do GitHub: a cada **git push**, um novo deploy acontece sozinho. A EC2 foi escolhida justamente por expor o processo completo, com mais aprendizado.

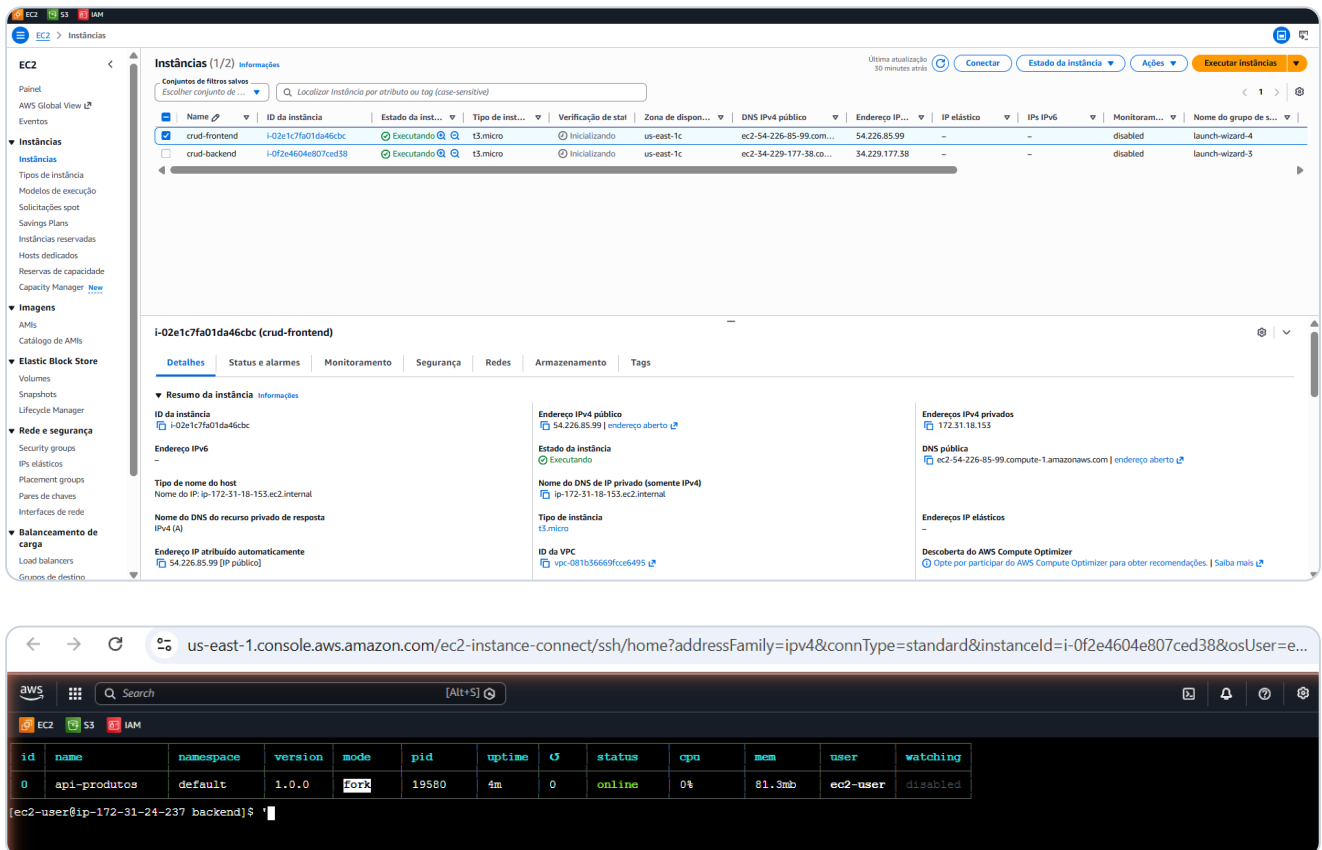


Figura 5.2 — As duas instâncias EC2 em execução no console da AWS e, abaixo, o PM2 mantendo a API no ar na VM do backend. *Descrição alternativa: painel da AWS exibindo as máquinas virtuais crud-frontend e crud-backend com estado "Executando"; terminal mostrando o processo api-produtos gerenciado pelo PM2 com status online.*

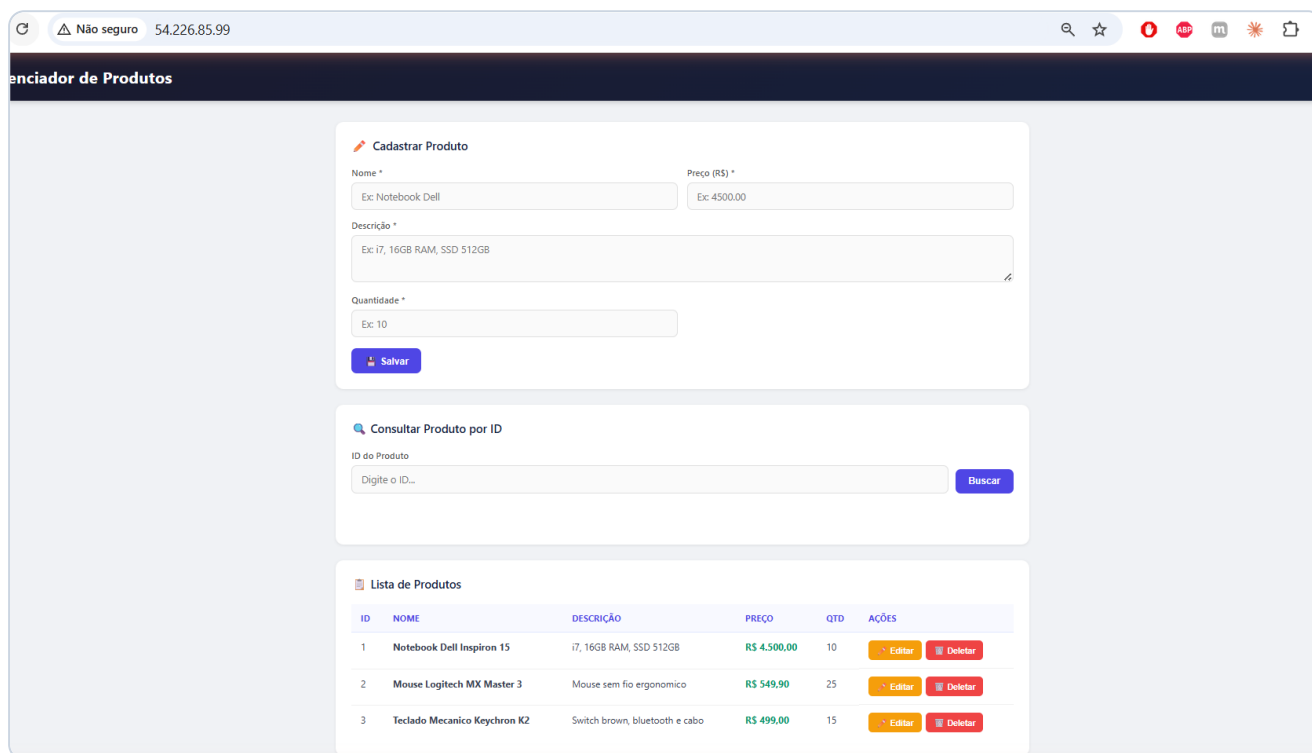


Figura 5.3 — O resultado final: aplicação acessível pela internet através do IP público da VM do front-end.
Descrição alternativa: navegador exibindo o Gerenciador de Produtos funcionando a partir do endereço público 54.226.85.99, com formulário e três produtos listados.

Testes e Qualidade de Software

Código que funciona hoje precisa continuar funcionando amanhã. Testes e ferramentas de qualidade são o seguro de vida de uma aplicação.

Testes manuais com Postman

Durante o desenvolvimento, cada rota da API foi testada manualmente com o **Postman**: enviamos requisições GET, POST, PUT e DELETE e conferimos os códigos de resposta (200, 201, 400, 404, 500) e o corpo em JSON. Esse ciclo rápido permitiu validar a API antes mesmo de o front-end existir.

Testes automatizados: o próximo nível

Testes manuais não escalam — a cada mudança, tudo precisaria ser reconferido à mão. Para o ambiente Node.js, as ferramentas indicadas são:

Ferramenta	Tipo	O que faz
Jest	Testes unitários	Verifica funções isoladas (ex.: validação de campos)
Supertest	Testes de integração	Simula requisições HTTP reais contra as rotas da API
ESLint	Análise estática	Apona erros e padrões ruins antes de o código rodar
Prettier	Formatação	Padroniza o estilo do código de toda a equipe

Um exemplo de teste de integração com Jest + Supertest para a rota de listagem:

```
backend/tests/produtos.test.js (exemplo)
const request = require('supertest');
const app = require('../index');

describe('GET /produtos', () => {
  test('deve retornar 200 e uma lista', async () => {
    const res = await request(app).get('/produtos');
    expect(res.statusCode).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });
});
```

Ambientes separados

Uma prática essencial de qualidade é nunca testar em produção. O ideal é manter pelo menos três ambientes: **desenvolvimento** (máquina local de cada dev), **homologação/teste** (uma réplica na nuvem onde as novidades são validadas) e **produção** (o que o

usuário final acessa). As variáveis do `.env` facilitam isso: basta trocar as credenciais para apontar para o banco de cada ambiente.

Integração contínua (CI)

Com **GitHub Actions**, os testes podem rodar automaticamente a cada push ou pull request: se algo quebrar, o GitHub bloqueia o merge e avisa a equipe. É a ponte entre versionamento (Capítulo 4) e qualidade — e o primeiro passo rumo ao *deploy contínuo*, em que a aprovação dos testes dispara a publicação automática na nuvem.

Conclusão e Reflexão

Mais do que um sistema de cadastro de produtos, este projeto foi um mergulho no ciclo de vida completo de um software: da primeira linha de código ao endereço público na internet.

Desafios enfrentados

- **CORS:** com front-end e back-end em servidores diferentes, o navegador bloqueava as requisições — entender e configurar o middleware `cors` foi o primeiro obstáculo real de "mundo distribuído".
- **Compatibilidade de versões:** o cliente do Supabase exigiu um ajuste (pacote `ws`) para funcionar em versões mais antigas do Node — registrado no histórico como um commit `fix`, mostrando o Git documentando a própria solução.
- **Segurança de credenciais:** aprender a manter chaves e senhas fora do repositório usando `.env` e `.gitignore`.
- **Deploy manual na EC2:** configurar Security Groups, conexão remota e PM2 exigiu paciência, mas revelou o que plataformas automatizadas escondem.

Aprendizados

O maior aprendizado foi perceber que **desenvolvimento de software vai muito além de programar**. Escolher ferramentas, organizar o ambiente, versionar com disciplina, testar e publicar são etapas tão importantes quanto o código em si — e são exatamente essas etapas que diferenciam um exercício de aula de um produto real.

O Node.js se mostrou uma porta de entrada generosa para o back-end: a familiaridade com o JavaScript do front-end reduziu a curva de aprendizado, e o ecossistema npm resolveu com poucas linhas problemas que pareciam grandes. Já o trabalho com Git e GitHub transformou a colaboração da equipe — conflitos que antes seriam "versões finais 2 (3) DEFINITIVA.zip" viraram branches, commits semânticos e pull requests revisáveis.

Por fim, o deploy na AWS desfez a aura de mistério da "nuvem": ela é, no fundo, o computador de outra pessoa — mas um computador que nunca desliga, acessível ao mundo, e que agora sabemos configurar.

📄 **Repositório do projeto:** github.com/matheusabporto/projeto3-cloud-crud-produtos